

Техническое задание для студентов

Производственная практика: 8-26 июня
Формат: командный проект, 2 команды
Защита: 26 июня

1. Название проекта

Система учета и аналитики заявок для учебного центра

2. Цель проекта

Разработать веб-приложение, которое позволяет учебному центру принимать, обрабатывать и анализировать заявки на обучение, консультации или мероприятия.

За время практики команда должна пройти полный цикл разработки: разобрать требования, спроектировать базу данных и программный интерфейс, реализовать минимальную рабочую версию, подготовить тестовые данные, написать инструкцию по запуску и показать проект на защите.

3. Пользователи системы

Роль	Что делает
Заявитель	Создает заявку и отслеживает ее статус
Менеджер учебного центра	Обрабатывает заявки, меняет статусы, оставляет комментарии
Администратор	Управляет справочниками, пользователями и базовыми настройками
Руководитель	Просматривает статистику и отчеты

4. Основные сценарии

1. Заявитель создает заявку: указывает тему, контактные данные, описание потребности, направление и желаемый формат.
 2. Менеджер видит список новых заявок и фильтрует их по статусу или направлению.
 3. Менеджер назначает ответственного или берет заявку в работу.
 4. Менеджер меняет статус заявки: Новая , В работе , Требуется уточнение ,
Согласована , Отклонена , Завершена .
 5. Менеджер добавляет комментарии к заявке.
 6. Администратор управляет справочниками: направления, форматы обучения, статусы.
 7. Руководитель смотрит дашборд: количество заявок, закрытые заявки, популярные направления, динамика по дням.
-

5. Функциональные требования

Обязательно

- Система должна иметь регистрацию и вход пользователей или заранее созданных тестовых пользователей.
- Система должна поддерживать минимум 3 роли: заявитель, менеджер, администратор/руководитель.
- Система должна позволять создавать заявку с темой, описанием, направлением, форматом, статусом и датой создания.
- Система должна позволять просматривать список заявок.
- Система должна поддерживать фильтрацию заявок по статусу и направлению.
- Система должна позволять менеджеру менять статус заявки.
- Система должна хранить данные в базе данных: PostgreSQL, MS SQL или другой согласованной с наставником СУБД.
- Система должна иметь пользовательский интерфейс. Команда может выбрать любой удобный подход: Blazor, React, Vue, Razor Pages, MVC, настольное приложение или другой вариант, если он позволяет показать основные сценарии.
- В репозитории должна быть инструкция по запуску проекта.
- В проекте должны быть базовые автоматические тесты хотя бы для ключевой бизнес-логики или сервисов.

Желательно

- Описание программного интерфейса через Swagger/OpenAPI или обычную текстовую документацию.
- Страница дашборда с простой аналитикой.

- Роли и ограничения доступа к страницам или методам программного интерфейса.
- Миграции БД и демонстрационные данные.
- Запуск через Docker Compose или понятная локальная инструкция запуска.
- Проверка изменений через запросы на просмотр кода или аналогичный командный процесс.

Дополнительно

- Экспорт заявок в CSV/Excel.
- Уведомления о смене статуса.
- Обновление списка заявок в реальном времени.
- Отдельный мобильный экран или настольный клиент.
- Автоматическая сборка и запуск тестов в системе непрерывной интеграции.
- Расширенная аналитика: графики по направлениям, статусам и динамике.
- Логирование через Serilog.
- Простая документация программного интерфейса.

Ограничения

- Не уходить в микросервисную архитектуру как обязательную часть проекта.
- Не делать сложные интеграции с внешними сервисами до готовности минимальной рабочей версии.
- Не хранить секреты, пароли, токены и строки подключения в репозитории.
- Не подменять реализацию только макетами без рабочей БД и основных сценариев.

6. Минимальный результат к защите

К 26 июня у каждой команды должно быть рабочее приложение, в котором можно:

- войти под тестовым пользователем;
 - создать заявку;
 - увидеть заявку в списке;
 - изменить статус заявки;
 - отфильтровать заявки;
 - открыть простую страницу статистики;
 - запустить проект по инструкции из репозитория.
-

7. Рекомендуемый стек

Серверная часть:

- C#;
- ASP.NET Core Web API;
- Entity Framework Core;
- PostgreSQL или MS SQL;
- Swagger/OpenAPI или текстовое описание программного интерфейса.

Пользовательский интерфейс:

- Любой удобный для команды вариант: Blazor, React, Vue, Razor Pages, MVC, настольное приложение или другой согласованный подход.
- Главное требование: интерфейс должен позволять показать основные сценарии без ручной правки данных в базе.

Инфраструктура:

- GitHub или другой Git-хостинг;
- GitHub Issues / Projects или простая доска задач;
- Docker Compose как дополнительная задача;
- GitHub Actions как дополнительная задача.

Тестирование:

- xUnit / NUnit / MSTest;
- минимум 3-5 автоматических тестов для сервисов, валидации или бизнес-логики.

8. Основные сущности

Минимальная модель данных:

Сущность	Основные поля
Пользователь	Идентификатор, имя, почта/логин, роль
Заявка	Идентификатор, тема, описание, направление, формат, статус, дата создания, дата изменения, автор, ответственный
Направление	Идентификатор, название

Сущность	Основные поля
Формат обучения	Идентификатор, название
Комментарий	Идентификатор, заявка, автор, текст, дата создания

Дополнительно можно добавить:

- StatusHistory: история изменения статусов;
- Attachment: файлы к заявке;
- AuditLog: аудит действий пользователей.

9. План работ

8 июня

- Сформировать команды.
- Создать репозиторий.
- Назначить роли внутри команды.
- Провести первичную декомпозицию задач.
- Согласовать стек и минимальную рабочую версию.

Контрольная точка: есть репозиторий, список задач, роли и черновая схема БД.

9 июня

- Уточнить задачи после первого вебинара.
- Начать реализацию каркаса серверной части и пользовательского интерфейса.
- Создать первые задачи и запросы на проверку изменений.

Контрольная точка: есть каркас проекта и первые задачи в работе.

10-13 июня

- Реализовать БД, модели, миграции.
- Реализовать регистрацию/логин или упрощенную авторизацию.
- Реализовать CRUD для заявок.
- Сделать первые экраны пользовательского интерфейса.

Контрольная точка: можно создать и посмотреть заявку, есть подключение к БД.

14-15 июня

- Доработать список заявок.
- Добавить фильтры, статусы и базовую валидацию.
- Подготовить вопросы к консультации.

Контрольная точка: команда понимает текущие блокеры.

16 июня

- Стабилизировать API.
- Стабилизировать программный интерфейс.
- Уточнить роли, доступы, тесты.

Контрольная точка: программный интерфейс стабилен, есть план по тестам и ролям.

17-20 июня

- Реализовать роли и ограничения доступа.
- Доработать пользовательский интерфейс.
- Добавить комментарии или историю статусов.
- Добавить базовые тесты.
- Начать страницу статистики.

Контрольная точка: минимальная рабочая версия почти готова, остаются отчеты, полировка, тесты и документация.

21-22 июня

- Стабилизировать минимальную рабочую версию.
- Исправить ошибки.
- Подготовить демонстрационные данные.
- Начать инструкцию по запуску и документацию.

Контрольная точка: приложение запускается по инструкции или близко к этому.

23 июня

- После третьего вебинара подготовить проект к защите.
- Проверить демонстрационный сценарий: вход -> создание заявки -> обработка -> аналитика.

Контрольная точка: команда показывает рабочий сценарий от начала до конца.

24-25 июня

- Провести финальное тестирование.
- Подготовить демо.
- Подготовить слайды или план выступления.
- Зафиксировать известные ограничения.
- Заморозить объем работ: новые функции добавлять только если они не ломают минимальную рабочую версию.

Контрольная точка: проект готов к защите.

26 июня

- Защита проектов.
 - Демонстрация рабочих сценариев.
 - Обсуждение архитектуры, командной работы и качества результата.
-

10. Ожидаемые артефакты

К защите команда должна подготовить:

- GitHub-репозиторий с исходным кодом;
 - инструкцию по запуску с описанием проекта и стека;
 - инструкцию по тестовым пользователям;
 - краткую схему БД или описание сущностей;
 - Swagger/OpenAPI или описание программного интерфейса;
 - список реализованных функций;
 - список известных ограничений;
 - 3-5 автоматических тестов или другой проверяемый набор автотестов;
 - демонстрационные данные для показа;
 - короткую документацию на 1-2 страницы в Markdown.
-

11. Критерии приемки

Проект принимается, если:

- репозиторий содержит исходный код и инструкцию запуска;
- основные сценарии работают на демонстрации без ручного исправления данных в БД;

- в БД есть минимум 3-5 тестовых заявок для демонстрации;
 - есть разделение ролей или хотя бы имитация ролей в минимальной рабочей версии;
 - есть минимум 3-5 автоматических тестов;
 - структура задач, запросов на проверку изменений или коммитов показывает вклад участников;
 - команда может объяснить архитектуру, схему БД и основные решения.
-

12. Формат защиты

Каждая команда показывает:

- какую проблему решали;
- какую минимальную рабочую версию выбрали;
- какие роли были в команде;
- архитектуру: пользовательский интерфейс, серверная часть, БД, тесты, инфраструктура;
- 1-2 главных пользовательских сценария вживую;
- что получилось хорошо;
- что не успели и почему;
- чему научились в командной разработке.

Рекомендуемая длительность выступления команды: 10-12 минут + 5 минут на вопросы.

13. Вопросы, к которым нужно быть готовыми

- Как вы разделили задачи внутри команды?
 - Какие решения приняли по архитектуре и почему?
 - Какие компромиссы были по объему работ?
 - Что произойдет, если в системе станет в 10 раз больше заявок?
 - Какой самый сложный баг или конфликт был в проекте?
 - Что бы вы улучшили, если бы была еще неделя?
 - Как проверяли качество результата?
-

14. Критерии оценки

Критерий	Вес
Рабочая минимальная версия	30%
Командная работа и Git-процесс	20%
Качество архитектуры и кода	20%
Удобство интерфейса и понятность пользовательских сценариев	10%
Тесты, документация, запуск	10%
Качество защиты и умение объяснить решения	10%

15. Если команда успевает быстрее

Можно взять задачи “со звездочкой”:

- экспорт CSV/Excel;
- уведомления о смене статуса в реальном времени;
- Docker Compose;
- GitHub Actions;
- расширенная аналитика и графики;
- аудит изменений статуса;
- Telegram/email-уведомления.

Главное правило: дополнительные функции не должны ломать минимальную рабочую версию.